

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#1)

**Apply modern simulation tools ModelSim and
Logisim to implement basic processor components**

Student Name: Zuha Fatima

Roll No: 24I-6534

Section: CE-A

Date performed: 18-08, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

// Write description of the task here  (max. five to seven lines)

In this task we write basic Verilog code for & gates and we also write there testbench in with we make an instant of our module and initialize the input combinations on which we want to check the output and after writing Verilog code and testbench we simulate it and verify our results through waveform .here we write code for AND,OR,NOT,NOR,XOR,XNOR through gate level modeling.

Code:

// Paste source code here 

```
AND
input a;
input b;
output c;

and al(c,a,b);

endmodule

module testbench();

reg x;
reg y;
wire z;
logicGates uut(x,y,z);

initial
begin
x=0;
y=0;
#50
x=0;
y=1;
#50
x=1;
y=0;
#50
```

```
x=1;
y=1;

end

endmodule
```

OR

```
module logicGates(a,b,c);
input a;
input b;
output c;
or al(c,a,b);
endmodule

module testbench();
reg x;
reg y;
wire z;
logicGates uut(x,y,z);
initial
begin
x=0;
y=0;
#50
x=0;
y=1;
#50
x=1;
y=0;
#50
x=1;
y=1;
end
endmodule
```

NAND

```
module Logic_gates(a,b,c);
input a;
input b;
output c;
nand al(c,a,b);
endmodule

module testbench();
reg x;
reg y;
wire z;
Logic_gates uut(x,y,z);
```

```
initial
begin
x=0; y=0;
#50
x=0; y=1;
#50
x=1; y=0;
#50
x=1; y=1;
end
endmodule

NOR
module Logic_gates(a,b,c);
input a;
input b;
output c;
nor al(c,a,b);
endmodule
module testbench();
reg x;
reg y;
wire z;
Logic_gates uut(x,y,z);
initial
begin
x=0; y=0;
#50
x=0; y=1;
#50
x=1; y=0;
#50
x=1; y=1;
end
endmodule

XOR
module Logic_gates(a,b,c);
input a;
input b;
output c;
xnor al(c,a,b);
endmodule
module testbench();
reg x;
reg y;
wire z;
Logic_gates uut(x,y,z);
```

```
initial
begin
x=0; y=0;
#50
x=0; y=1;
#50
x=1; y=0;
#50
x=1; y=1;
end
endmodule

XNOR
module Logic_gates(a,b,c);
input a;
input b;
output c;
xnor al(c,a,b);
endmodule

module testbench();
reg x;
reg y;
wire z;
Logic_gates uut(x,y,z);
initial
begin
x=0; y=0;
#50
x=0; y=1;
#50
x=1; y=0;
#50
x=1; y=1;
end
endmodule

NOT
module Logic_gates(a,c);
input a;
//input b;
output c;
not al(c,a);
endmodule

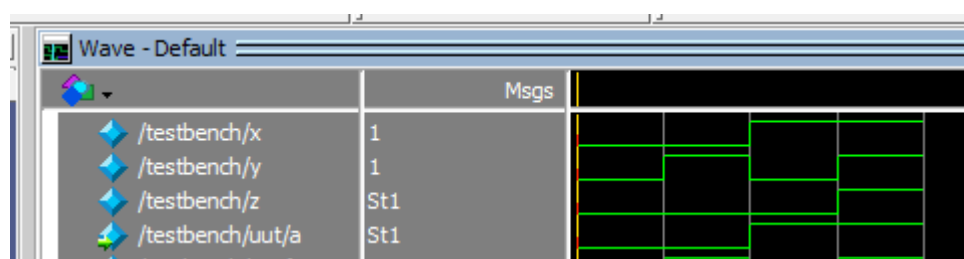
module testbench();
reg x;
//reg y;
wire z;
Logic_gates uut(x,z);
initial
```

```
begin
x=0;// y=0;
//#50
//x=0; y=1;
#50
x=1;// y=0;
//#50
//x=1; y=1;
end
endmodule
```

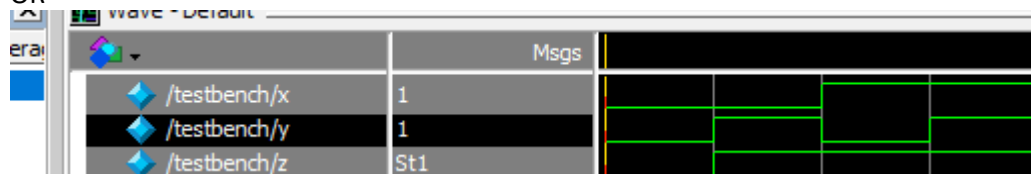
Output:

// Import output here

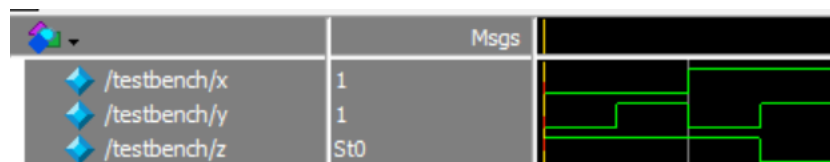
AND



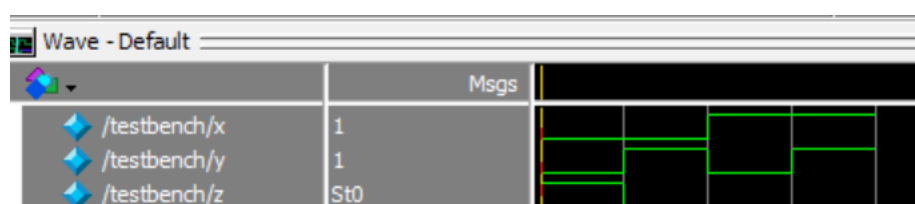
OR



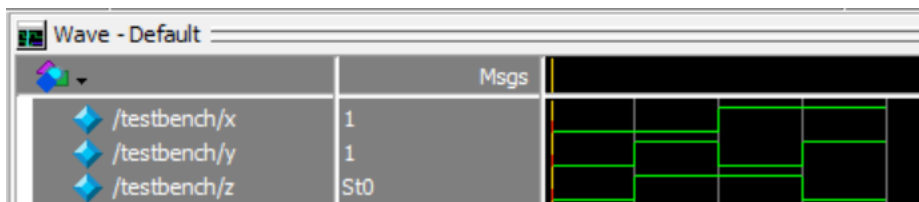
NAND



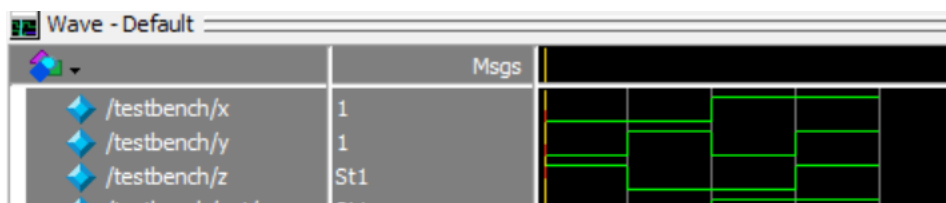
NOR



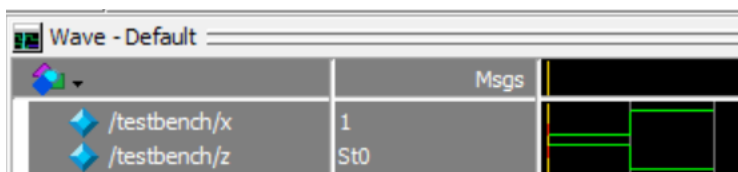
XOR



XNOR



NOT



Task 2:

Description:

// Write description of the task here  (max. five to seven lines)

In this task we write basic Verilog code for & gates and we also write there testbench in with we make an instant of our module and initialize the input combinations on which we want to check the output and after writing Verilog code and testbench we simulate it and verify our results through waveform .here we write code for AND,OR,NOT,NOR,XOR,XNOR through data flow level modeling.

Code:

// Paste source code here 

```
AND
module Logic_gates(a,b,c);
input a;
input b;
output c;
assign c=a&b;
endmodule

module testbench();
reg x;
reg y;
wire z;
Logic_gates uut(x,z);
initial
begin
x=0;y=0;
#50
x=0; y=1;
#50
x=1; y=0;
#50
x=1; y=1;
end
endmodule

OR
module Logic_gates(a,b,c);
input a;
input b;
output c;
assign c=a|b;
```

```
endmodule
module testbench();
  reg x;
  reg y;
  wire z;
  Logic_gates uut(x,z);
  initial
  begin
    x=0;y=0;
    #50
    x=0; y=1;
    #50
    x=1; y=0;
    #50
    x=1; y=1;
  end
endmodule

NAND
module Logic_gates(a,b,c);
  input a;
  input b;
  output c;
  assign c=~(a&b);
endmodule

module testbench();
  reg x;
  reg y;
  wire z;
  Logic_gates uut(x,z);
  initial
  begin
    x=0;y=0;
    #50
    x=0; y=1;
    #50
    x=1; y=0;
    #50
    x=1; y=1;
  end
endmodule

NOR
module Logic_gates(a,b,c);
  input a;
  input b;
  output c;
  assign c=~(a|b);
```

```
endmodule
module testbench();
  reg x;
  reg y;
  wire z;
  Logic_gates uut(x,z);
  initial
  begin
    x=0;y=0;
    #50
    x=0; y=1;
    #50
    x=1; y=0;
    #50
    x=1; y=1;
  end
endmodule

XOR
module Logic_gates(a,b,c);
  input a;
  input b;
  output c;
  assign c=a^b;
endmodule

module testbench();
  reg x;
  reg y;
  wire z;
  Logic_gates uut(x,z);
  initial
  begin
    x=0;y=0;
    #50
    x=0; y=1;
    #50
    x=1; y=0;
    #50
    x=1; y=1;
  end
endmodule

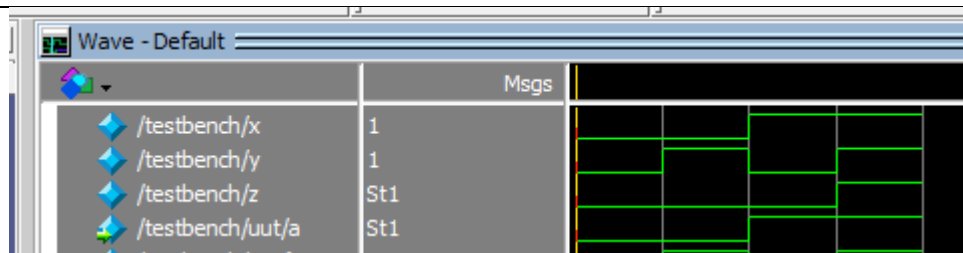
XNOR
module Logic_gates(a,b,c);
  input a;
  input b;
  output c;
  assign c=a~^b;
endmodule
```

```
module testbench();
  reg x;
  reg y;
  wire z;
  Logic_gates uut(x,z);
  initial
  begin
    x=0;y=0;
    #50
    x=0; y=1;
    #50
    x=1; y=0;
    #50
    x=1; y=1;
  end
endmodule
NOT
module Logic_gates(a,c);
  input a;
  //input b;
  output c;
  assign c=~a;
endmodule
module testbench();
  reg x;
  //reg y;
  wire z;
  Logic_gates uut(x,z);
  initial
  begin
    x=0;// y=0;
    // #50
    //x=0; y=1;
    #50
    x=1;// y=0;
    // #50
    //x=1; y=1;
  end
endmodule
```

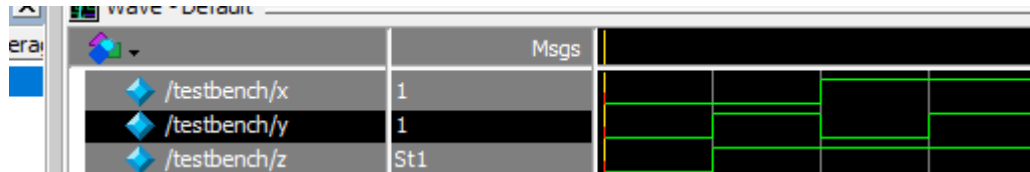
Output:

// Import output here 🖱

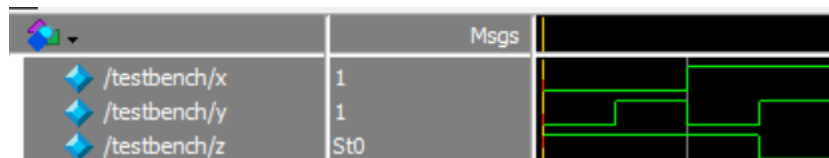
AND



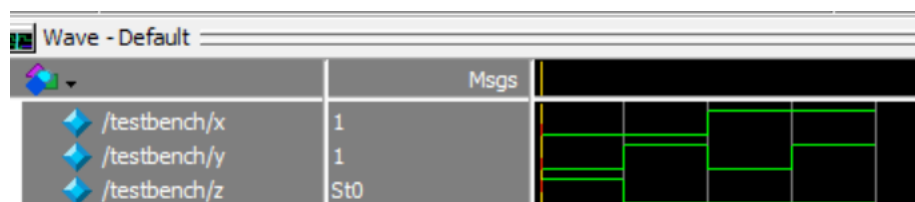
OR



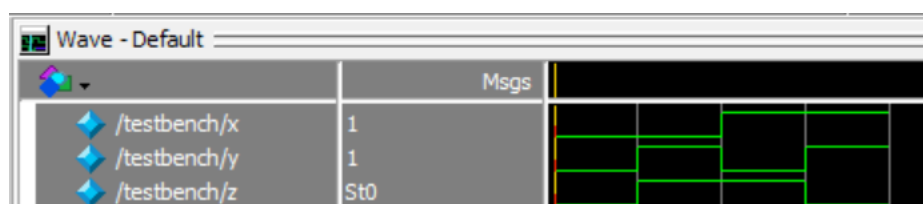
NAND



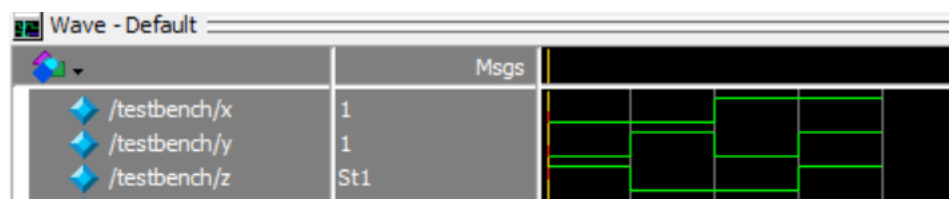
NOR



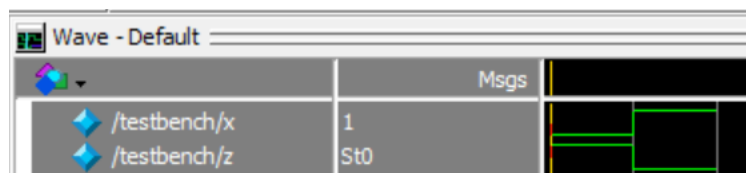
XOR



XNOR



NOT



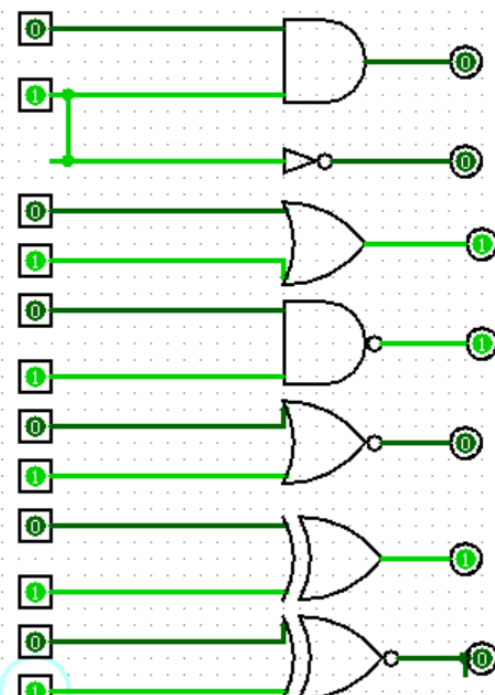
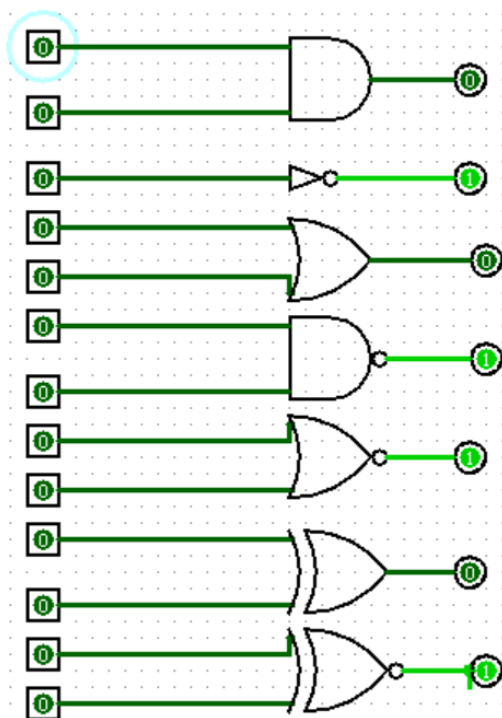
Task 3:

Description:

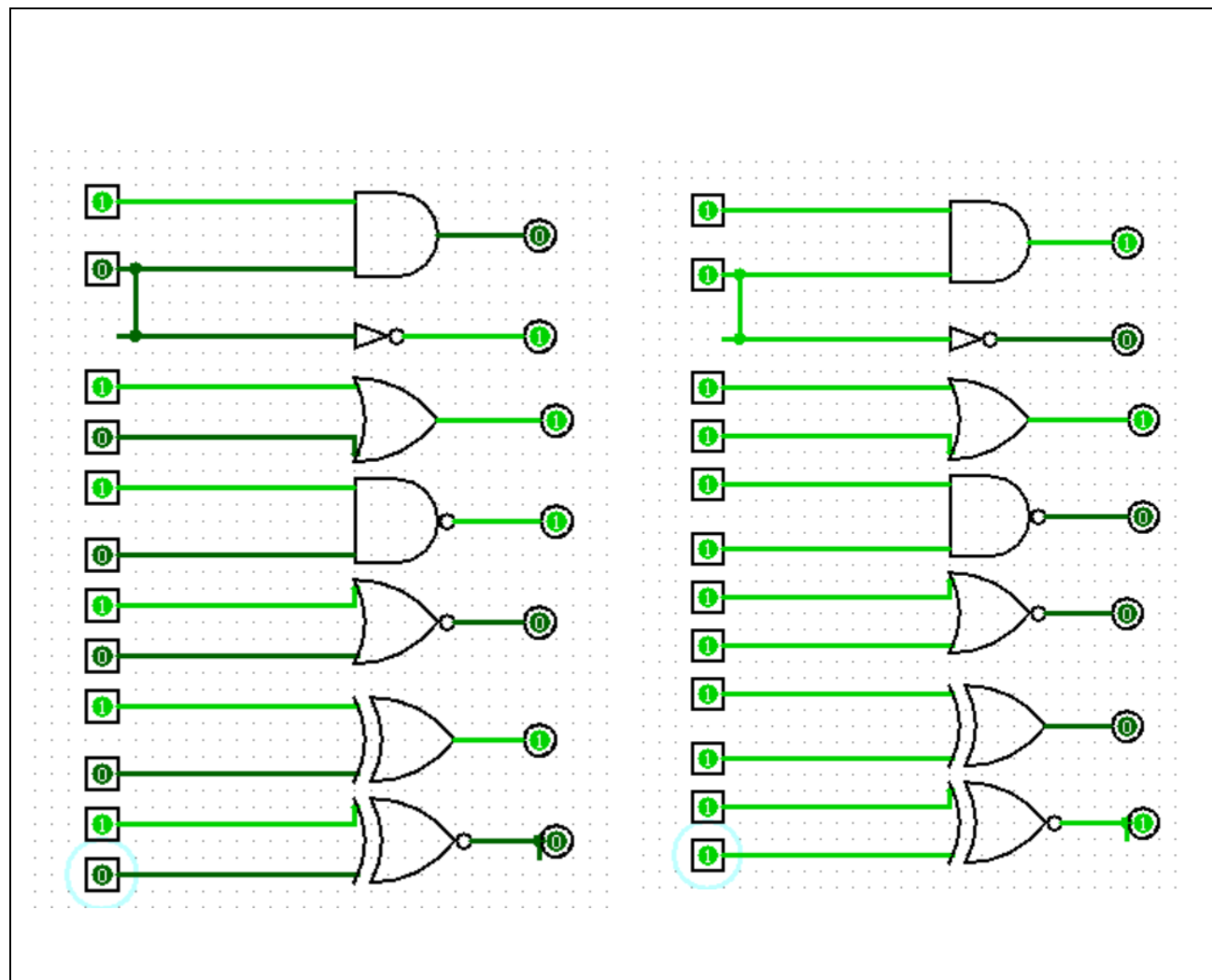
// Write description of the task here (max. five to seven lines)

In this lab task, we test the working of basic logic gates using Logisim. We make different logic gates and give them different inputs. The main gates used are AND, OR, NOT, NAND, NOR, XOR, and XNOR. We observe the output for each input combination. This helps us understand how basic logic gates work.

Code:



// Paste source code here



Task 4:

Description:

// Write description of the task here 🖱️ (max. five to seven lines)

In this lab, we design the given circuit using Verilog code. We use gate-level and data-flow modeling for the circuit. The circuit has two inputs and one output. NOT, AND, and OR gates are used to make the circuit. We write a testbench to give different inputs to the circuit. The output is checked to verify that the circuit is working correctly.

Code:

// Paste source code here 🖱️

Gate level :
module Lab1_circuit(a,b,g);
input a;

```
input b;
output g;
wire c,d,e,f;
not (c,a);
not (d,b);
and (e,c,b);
and (f,a,d);
or (g,e,f);
endmodule
module Lab1_circuit_tb();
  reg x;
  reg y;
  wire z;
  Lab1_circuit uut(x,y,z);
  initial
  begin
    x=0;y=0;
    #50
    x=0; y=1;
    #50
    x=1; y=0;
    #50
    x=1; y=1;
  end
endmodule
Data flow level:
module Lab1_circuit(a,b,g);
input a;
input b;
output g;
wire c,d,e,f;
assign c=~a;
assign d=~b;
assign e=c&b;
assign f=a&d;
assign g=e|f;
endmodule
module Lab1_circuit_tb();
  reg x;
  reg y;
```



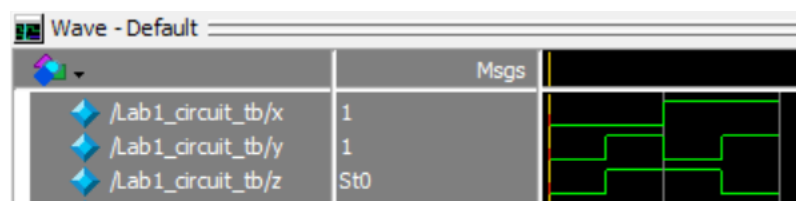
```

wire z;
Lab1_circuit uut(x,y,z);
initial
begin
x=0;y=0;
#50
x=0; y=1;
#50
x=1; y=0;
#50
end
endmodule

```

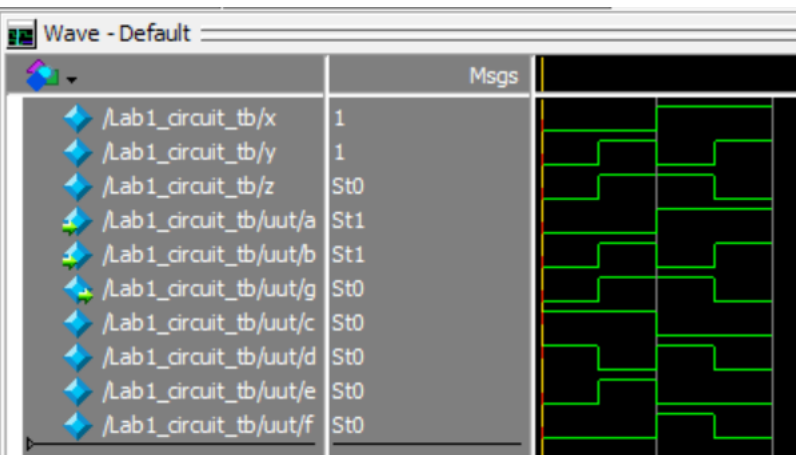
Output:

// Import output here 



Gate level

Data flow

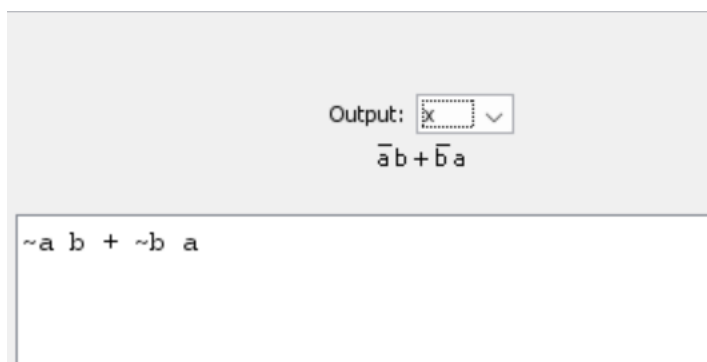


Task 5:

Description:

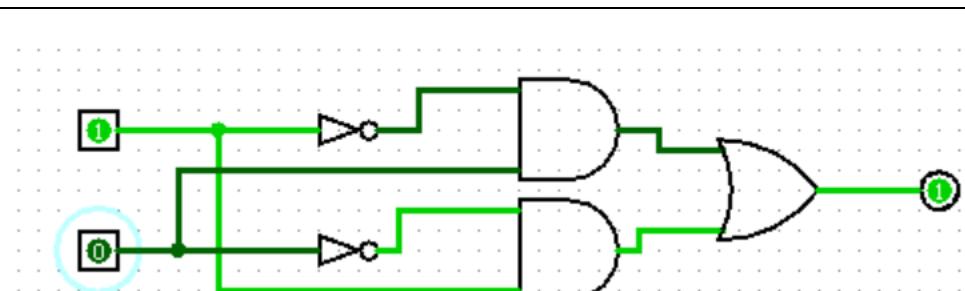
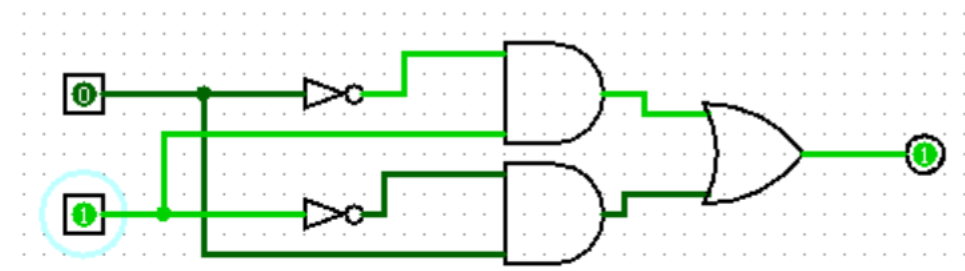
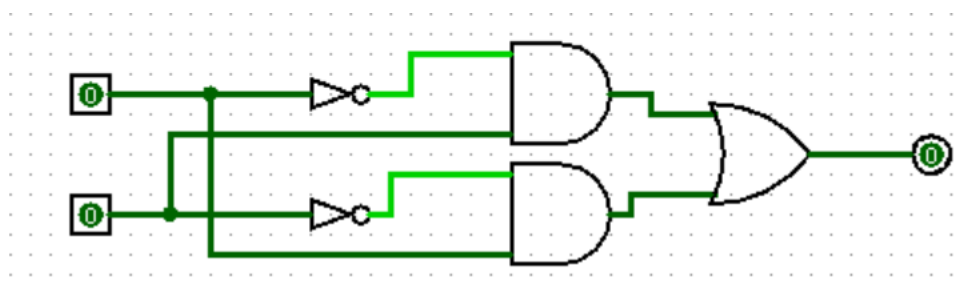
In this lab, we test the given circuit using Logisim. We make the same circuit in Logisim using basic logic gates. Different input combinations are given to the circuit. We observe the output for each input. The Logisim results are compared with the Verilog results in diagram as well as in truth table. This helps us check that the circuit is working correctly.

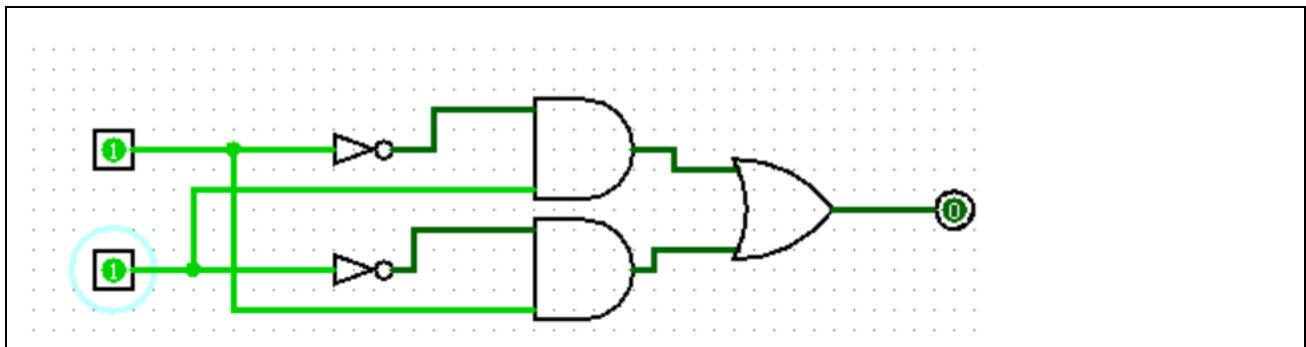
a	b	x
0	0	0
0	1	1
1	0	1
1	1	0



Output:

// Import output here 





Task 6:

Description:

// Write description of the task here 🖱 (max. five to seven lines)

In this lab task, we find the 1's complement of a 4-bit number. It means changing 0 into 1 and 1 into 0. We use gate-level and data-flow modeling to make the circuit. A testbench is used to check the results in this task I check the 5 test cases,

Code:

// Paste source code here 🖱

GATE LEVEL

```
module Lab1_complem(a,b);  
input [3:0] a;  
output [3:0] b;  
not (b[0], a[0]);  
not (b[1], a[1]);  
not (b[2], a[2]);  
not (b[3], a[3]);
```

```
endmodule  
module Lab1_complem_tb;  
reg [3:0] a;  
wire [3:0] b;
```

```
Lab1_complem uut(a, b);  
initial  
begin  
    a = 4'b0000;  
    #50;  
    a = 4'b0001;  
    #50;  
    a = 4'b0101;  
    #50;  
    a = 4'b1010;  
    #50;  
    a = 4'b1111;  
end  
endmodule
```

DATA FLOW LEVEL

```
module Lab1_complem(a,b);  
input [3:0] a;  
output [3:0] b;  
assign b=~a;  
endmodule  
module Lab1_complem_tb;  
reg [3:0] a;  
wire [3:0] b;  
Lab1_complem uut(a, b);  
initial  
begin  
    a = 4'b0000;  
    #50;  
    a = 4'b0001;  
    #50;  
    a = 4'b0101;  
    #50;  
    a = 4'b1010;  
    #50;  
    a = 4'b1111;  
end  
endmodule
```

Output:

// Import output here 🖱

GATE LEVEL

DATA FLOW LEVEL

Signal	Value	0000	0001	0101	1010	1111
/Lab1_complem_tb/a	0000	0000	0001	0101	1010	1111
/Lab1_complem_tb/b	1111	1111	1110	1010	0101	0000

Signal	Value	0000	0001	0101	1010	1111
/Lab1_complem_tb/a	1111	0000	0001	0101	1010	1111
/Lab1_complem_tb/b	0000	1111	1110	1010	0101	0000
/Lab1_complem_tb/...	1111	0000	0001	0101	1010	1111